서비스 종료 Unity 게임의 로컬 데이터 이식과 상태 복원 분석

Android 기기 간 이식 기록과 바이너리 비교에 기반한 탐색적 사례 연구

작성자 양다빈 작성일 2026년 10월 3일
과제 10 AI와 함께 쓰는 첫 논문

초록

서비스 종료된 Unity 기반 Android 게임의 리소스와 진행 상태 보존 가능성을 조사함. 기존 수행 기록과 PC 백업 자료, 탐색기·게임 실행·PowerShell 캡처를 대상으로 사후 가설을 설정하고 사례를 재분석함. AssetStudio를 통한 리소스 추출과 Shizuku의 ADB 권한 위임 후 기기 간 이식 및 상태 복원 기록을 확인함. S22+와 Tab S6 Lite의 offline_* 9개 파일 쌍에서 8개는 크기가 다르고 1개는 동일함. offline_flag는 두 자료 모두 160 B이나 SHA-256이 다르며, 동일 위치 10개 블록 및 전체 교차 비교에서 공통 16바이트 블록이 검출되지 않음. offline_userdata의 엔트로피는 각각 7.7202와 7.9164 bits/byte로 기록됨. 결과는 해당 환경에서의 로컬 상태 재사용 가능성을 뒷받침하며, 특정 암호화 방식이나 플랫폼 취약점의 근거로 확장할 수 없음.

주제어 Android Unity AssetBundle 로컬 데이터 데이터 이식 Shannon entropy

연구 요약

항목	범위
관심 분야	모바일 애플리케이션 보안 및 서비스 종료 게임의 데이터 보존
연구 유형	기존 수행 기록과 캡처를 활용한 탐색적 사례 연구
주요 자료	과거 이식 1건의 기록, PC 자료 집합 2개, offline_* 파일 9쌍
주요 결론	기기 간 상태 복원 기록과 바이너리 차이 확인
재현 범위	캡처 전사 수치의 검산 가능. 원본 바이너리를 이용한 독립 재계산은 별도 자료 필요

1 서론

1 1 연구 배경

기존 Android 스마트폰에 서비스 종료된 오프라인 Unity 게임이 남아 있었으나, iOS 기기로 변경하면서 공식적인 계정 이전이 불가능해짐. 이미지와 음성을 보존하기 위해 게임 데이터 경로에서 Bundle 파일을 확보한 뒤, 기존 진행 상태를 다른 Android 기기에서도 사용할 수 있는지 조사함. 리소스 열람 가능성, 플랫폼 권한의 전제조건, 상태 복원 결과를 구분하여 분석함.

1 2 가설과 측정 기준

최종 가설 H1 동일 게임 APK를 새로 설치한 Android 기기에 기존 게임 로컬 데이터를 적용하면, 게임에 표시되는 계정 및 진행 상태가 신규 상태에서 기존 상태로 바뀐다.

바꾸는 조건은 기존 데이터의 적용 여부이며, 관찰 결과는 게임 실행 가능 여부와 표시되는 계정·진행 상태임. 상태 복원은 과거 수행 기록의 기존 상태 복원 진술로 판정하고, 프로필·퀘스트 화면은 표시 상태의 보조 증적으로 사용함. 동일 Player ID의 전후 화면이 연결되지 않아 동일 계정 판정의 독립 검증에는 제약이 있음.

반증 조건은 적용 후 실행 실패, 신규 상태 유지 또는 기존 상태와 다른 결과임. 이러한 결과도 그대로 기록하는 설계임. 현재 H1은 결과를 확인한 뒤 정리한 사후 가설이며, 사전 등록된 통제 실험으로 제시하지 않음.

1 3 검토한 후보 주제

아래 후보는 과제 문서화 단계에서 기존 자료로 검증할 수 있는 범위를 비교한 것임. 과거 실제 선택 이력으로 소급하지 않음.

후보	측정 가능성 및 선정 또는 제외 이유
로컬 데이터 적용과 상태 복원	최종 선정. 수행 기록과 복사·게임 상태 화면을 연결할 수 있음.
16바이트 정렬과 AES 사용 여부	제외. 길이 정렬·높은 엔트로피만으로 알고리즘을 식별할 수 없음.
Shizuku 사용과 Android 취약점 여부	제외. 사용자 권한 위임 기록만으로 권한 통제 결함을 검증할 수 없음.
기기 변경과 데이터 무결성 보호 여부	제외. 파일 변경 실험과 오류·검증 코드 자료가 없어 보호 기능을 판정할 수 없음.

H1은 보존이라는 실제 목적에 직접 연결되고 반대 결과를 정의할 수 있어 선정함. 이식 후 표시 상태와 바이너리 동일성을 별개의 질문으로 나누어, 복원 성공을 곧바로 바이트 단위 동일성으로 해석하지 않음.

2 방법

2 1 연구 대상과 자료 범위

대상은 패키지 jp.co.koeitecmo.zone의 Unity 기반 Android 게임임. 조사 경로는 /Android/data/jp.co.koeitecmo.zone이며 앱 전용 외부 저장소에 해당함. 앱 내부 사설 저장소 /data/user/0/<패키지명>과 구분함. Android 11 저장소 제한의 일반 구조는 공식 문서 [1]에 근거함.

자료 또는 도구	내용
과거 이식 기록	기존 스마트폰 → 신규 Android 태블릿. 복원 성공 기록 1건.
01 자료	01_S22Plus_Before_Migration. S22+ 마이그레이션 전 확보 자료.
02 자료	02_TabS6Lite_Source. Tab S6 Lite 원본 자료.
03 자료	03_S22Plus_After_Migration 폴더 존재 확인. 정량 비교에는 사용하지 않음.
비교 경로	01의 패키지 폴더 하위와 02 루트를 동일 상대경로 기준으로 정렬.
도구	AssetStudio.net472.v0.16.47, PowerShell, Shizuku, zArchiver
시간 범위	과거 수행·수집 일시는 특정되지 않음. 제공 캡처 파일명은 2026.10.01~02 범위이며, 과제 문서화·검산일은 2026.10.03임.

2 2 조건과 표본 단위

구분	설계와 실제 기록
바꾼 조건	과거 이식: 기존 데이터 적용 여부. PC 비교: 01·02 자료 집합.
의도한 고정 조건	동일 게임 APK 설치와 동일 패키지 경로. APK 해시 및 버전 동일성은 자료에서 확인되지 않음.
통제되지 않은 조건	OS·보안 패치·앱 실행 상태·수집 시점·범위·네트워크 상태 및 계정 상태.
표본 수	자료 집합 2개, 이름을 맞춘 파일 9쌍, userdata 엔트로피 2개, flag 동일 위치 블록 10쌍.
반복 횟수	과거 이식 성공 기록 1건. 독립적인 10회 반복 이식 실험은 없음.
블록 비교 단위	flag 160 B를 16 B씩 분할한 10개 블록. 교차 비교는 10×10=100쌍이며 독립 실험 횟수가 아님.

현재 자료를 재분석하는 설계로 범위를 고정함. 결과를 맞추기 위해 파일 쌍이나 불일치 결과를 제외하지 않으며, 공개 데이터셋은 사용하지 않음.

2.3 수행 절차와 분석 방법

기존 스마트폰에서 Bundle을 PC로 가져와 AssetStudio로 로드·분석하고 이미지·일러스트·캐릭터 음성을 추출한 기록을 검토함. 태블릿에 동일 게임 APK를 설치하고 초기 실행 환경을 생성한 후, 일반 파일 관리자에서 제한된 /Android/data 파일 작업을 Shizuku와 zArchiver를 통해 수행함. 기존 데이터 폴더를 복사 가능한 위치로 옮겨 태블릿의 동일 게임 데이터 영역에 덮어쓴 뒤 실행 결과를 확인함.

Shizuku 공식 설명은 ADB 또는 root로 시작한 서비스가 앱의 시스템 API 호출을 중개하는 구조를 설명함 [2]. 이 사례의 수행 기록은 ADB 수준 권한 위임에 해당하며, root 사용이나 플랫폼 권한 결함을 입증하는 자료는 없음.

PC 비교에서는 이름을 맞춘 offline_* 파일의 크기, SHA-256, 바이트 분포의 Shannon entropy, 크기의 16바이트 배수 여부와 블록 일치 여부를 조사한 기존 출력을 검토함. 엔트로피는 $H = -\Sigma\, p(b)\, \log_2 p(b)$로 정의하며 단위는 bits/byte임. 유한 길이 표본의 분포를 나타내는 값으로, 값만으로 특정 암호화 방식을 식별하지 않음 [3].

측정값	계산 또는 판정 방법
파일 크기	바이트 수 N. 차이는 N02 − N01로 계산함.
SHA-256	파일 전체의 해시 비교. 불일치는 바이트 내용 차이를 나타냄.
엔트로피	256개 가능한 바이트 값의 빈도로 p(b)를 산출하여 계산함.
16바이트 정렬	N mod 16 = 0인지 검사함. 암호 처리의 필요충분조건이 아님.
동일 위치 블록	각 flag를 16 B로 분할하고 같은 인덱스의 바이트열을 비교함.
전체 교차 블록	01의 각 블록과 02의 모든 블록을 비교하여 공통 블록 유무 확인함.
복원 상태	과거 수행 기록과 현재 캡처의 표시 상태를 구분하여 판독함.

2.4 기록 보존과 재현 수준

과거 수행 당시 전체 과정을 스크린샷으로 기록하지 않아 일부 단계는 당시 기록과 현재 확인 가능한 파일 구조 및 실행 결과를 기반으로 정리함.

패키지의 data CSV는 PowerShell 캡처와 비교 문서의 수치를 전사한 관찰표이며 원본 게임 바이너리가 아님. 증적 PNG와 출처 문서를 함께 보존하고, 검산 스크립트 및 원본 파일을 제공받았을 때 사용할 읽기 전용 분석 스크립트를 구분함. 게임 화면의 계정 식별정보는 공개본에서 마스킹함. 작업 과정의 추가 독립 실험이나 사전 설계 변경 기록은 생성하지 않음.

3 결과

3 1 데이터 경로와 Unity 리소스

자료 집합 01·02·03의 폴더 구분과 패키지 files 하위 구조를 확인함. files에는 bundles, com.unity.addressables, fc, il2cpp, local, Unity와 regist 등이 표시됨. local에서 비교 대상 offline_* 이름을 확인하고, bundles에서 Bundle 파일 목록을 확인함. AssetStudio에서 리소스를 추출한 결과는 과거 수행 기록에 근거하며 아래 탐색기 화면을 추출 성공 화면으로 대체하지 않음.

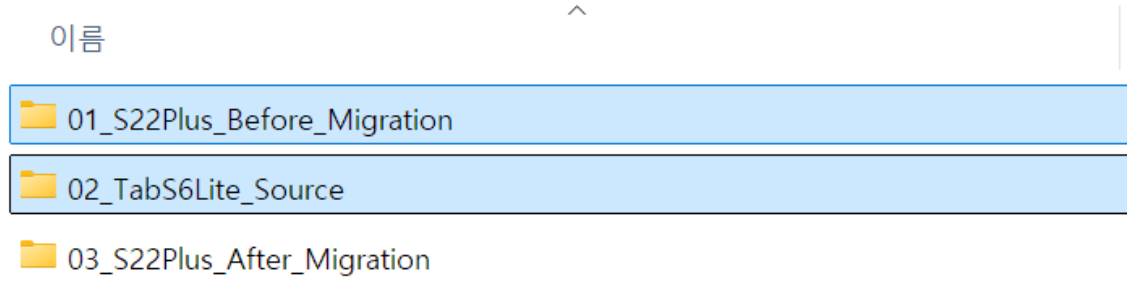


그림 5. PC 백업 자료 집합 01·02·03의 폴더 구분

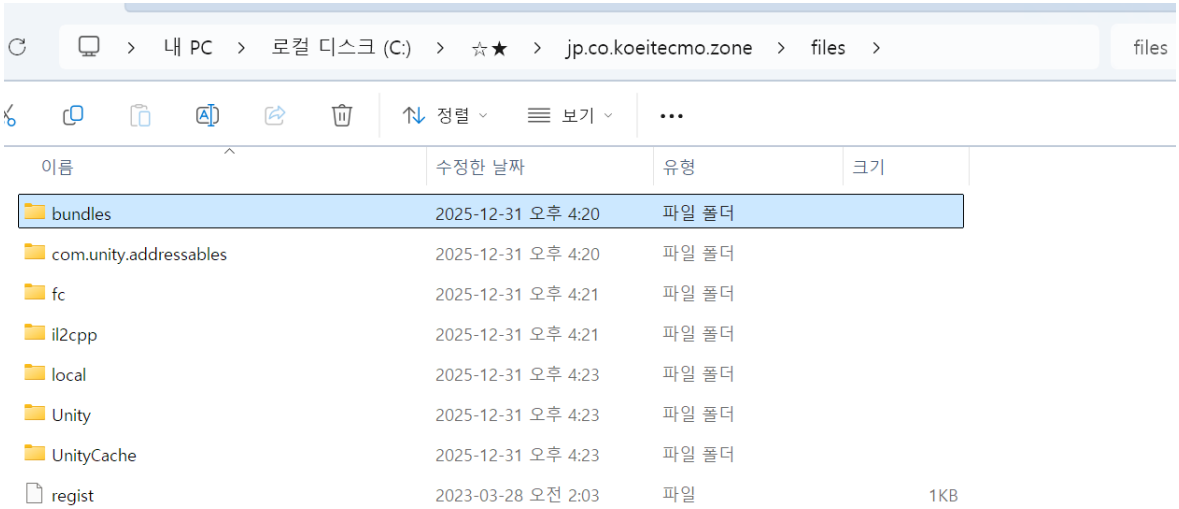


그림 6a. jp.co.koeitecmo.zone의 files 하위 구조

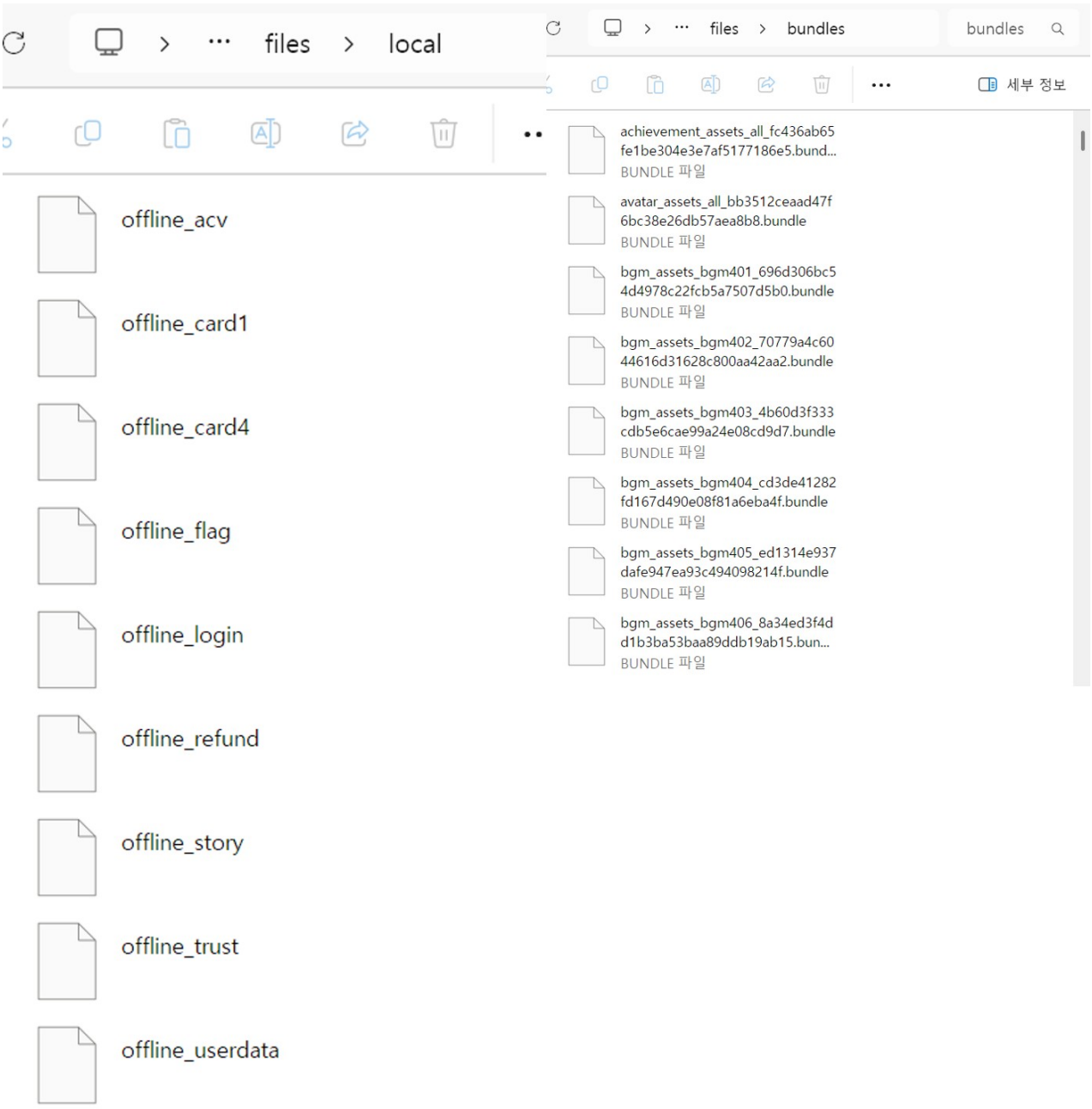


그림 6b, 6c. local의 offline_* 목록과 bundles 파일 목록

3.2 접근 권한과 데이터 적용

과거 기록에서 일반 파일 관리자의 Android/data 접근·복사 제한과 Shizuku를 통한 zArchiver의 ADB 수준 권한 위임 후 파일 작업을 확인함. 해당 기록을 Android 11 이상 저장소 정책과 대조하되, 과거 OS·targetSdk와 오류 메시지가 특정되지 않아 정책 적용 조건 전체를 재현한 것으로 평가하지 않음 [1][2].

추가 탐색기 캡처에는 S22+ 대상의 files 폴더 병합 대화상자와 bundles 복사 진행이 표시됨. 이는 파일 작업의 직접 증적이나 과거 스마트폰 → 태블릿 이식과 복사 방향이 다르므로 같은 시행의 전후 화면으로 합치지 않음.

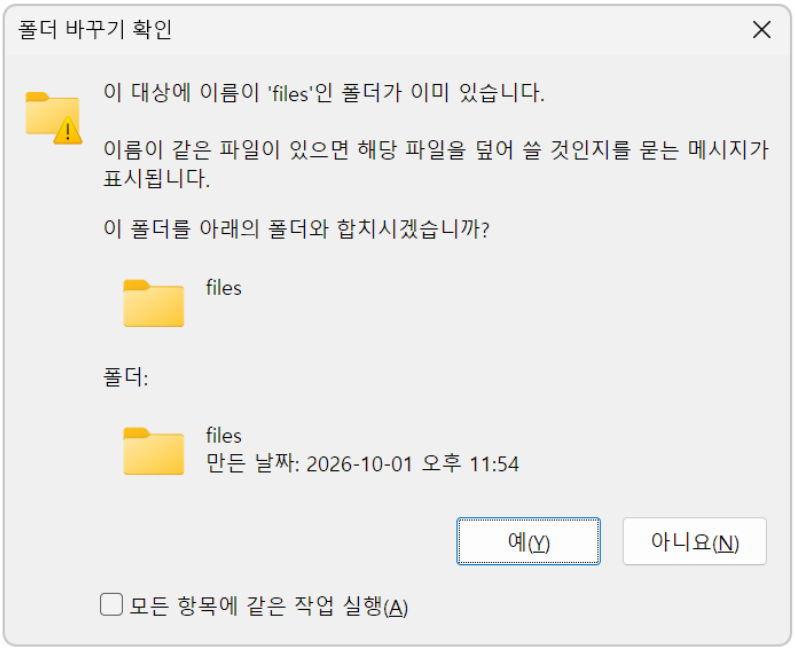


그림 7. 동일 이름의 files 폴더 병합 확인 대화상자

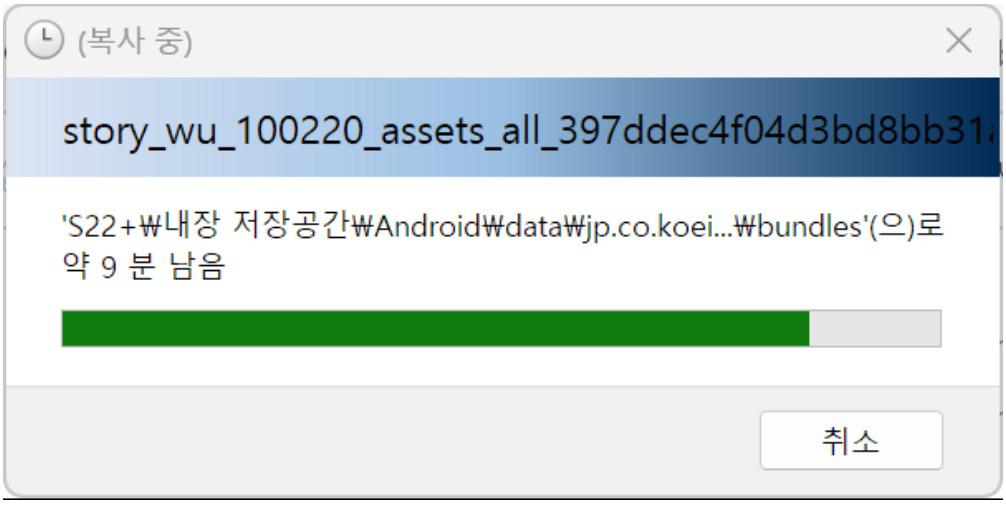


그림 8. S22+의 Android/data 게임 경로를 대상으로 한 bundles 복사 진행

AssetStudio 실행·추출, 일반 파일 관리자 제한 및 Shizuku·zArchiver 권한 화면은 현재 수록 캡처에 포함되지 않음. 해당 단계의 근거 수준은 수행 기록임.

3.3 게임 실행과 표시 상태

과거 수행 기록에서는 신규 태블릿에서 기존 계정 및 진행 상태가 정상적으로 복원되어 실행됨. 추가 프로필 화면에서 랭크 98, 진행 화면에서 Normal 252/252와 Hard 104/104 등을 확인함. 화면의 기기 대응과 이식 전후 동일 Player ID 대조가 연결되지 않아, 캡처만으로 전체 복원 과정을 독립 입증하지 않음.



그림 9. 랭크 98이 표시된 프로필 화면. Player ID 등 식별정보 마스킹



그림 10. 퀘스트 Normal 252/252, Hard 104/104 및 캐릭터별 스토리 진행 상태

H1은 해당 1건의 수행 기록에서 지지됨. 이식 성공률이나 다른 기기·OS에서의 복원 가능성은 산출하지 않음.

3 4 PC 자료 집합과 파일 크기

비교 문서에는 01 35개, 02 5,304개 파일이 기록됨. 수집 범위가 동일하다고 확인되지 않아 파일 수 차이를 이식으로 인한 증가로 해석하지 않음. unity.ver, global-metadata.dat, Ionic.Zip.dll-resources.dat, mscorlib.dll-resources.dat 및 Unity Analytics config의 동일 경로 해시 일치가 기록됨. 01의 일부 UnityShaderCache 및 Analytics ArchivedEvents는 02의 동일 상대경로에서 확인되지 않음.

파일	01 B	02 B	02 − 01 B
offline_acv	30,416	30,688	+272
offline_card1	224	8,144	+7,920
offline_card4	224	1,648	+1,424
offline_flag	160	160	0
offline_login	1,184	1,168	-16
offline_refund	224	240	+16
offline_story	3,184	227,456	+224,272
offline_trust	52,416	52,448	+32
offline_userdata	720	2,832	+2,112

9쌍 중 8쌍은 크기가 다르며 offline_flag 1쌍은 동일함. 18개 파일 크기와 9개 차이는 모두 16 B의 배수임. 증가만 발생한 것이 아니며 offline_login은 16 B 작음. 크기 동일성과 내용 동일성을 구분함.

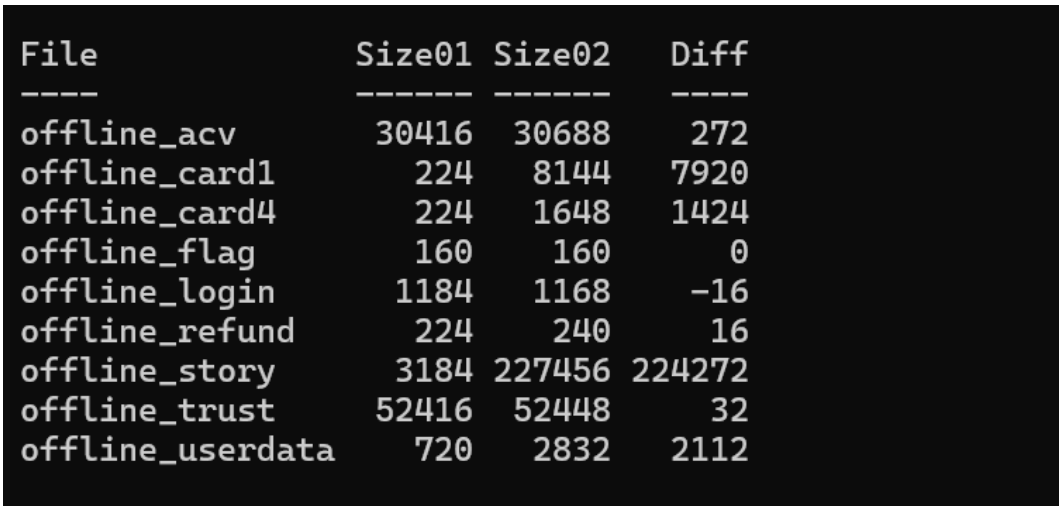


그림 1. offline_* 9개 파일 쌍의 크기와 차이 출력

3 5 해시 엔트로피와 Analytics 구조

자료	userdata 크기 B	엔트로피 bits/byte
01 S22+	720	7.7202
02 Tab S6 Lite	2,832	7.9164

offline_userdata의 SHA-256은 서로 다름. ZIP·gzip·SQLite 대표 시그니처와 의미 있는 ASCII 평문 구조는 식별되지 않은 것으로 기록됨. offline_acv·story·trust는 엔트로피가 8에 가까운 것으로 기록되었으나 정확한 수치는 제공되지 않아 별도 수치표를 만들지 않음.

offline_userdata SHA-256 01
84EBE0A04812D24D3AEB5D226ABD6477A6C694BEAAFE918E4E04943D946D99D4

offline_userdata SHA-256 02
6C4109B34F6F245BB2E2186344A7A6D54BF48C010A627445454AAF16FDF47E54

높은 엔트로피는 바이트 분포가 비교적 분산되어 있음을 보여줌. 길이가 서로 다른 표본이며 압축·인코딩·암호화 등 여러 처리 가능성이 있으므로 AES 사용이나 기기별 키를 식별하는 근거로 사용하지 않음. 실험 연구 [3]의 악성코드 대상 결과나 임계값을 게임 상태 데이터에 그대로 적용하지 않음.

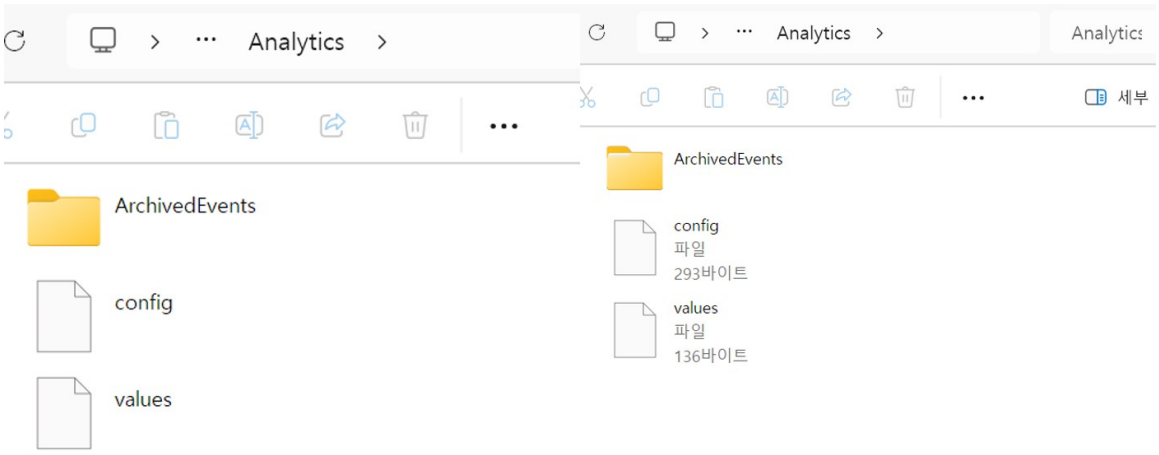


그림 11a, 11b. Unity Analytics의 ArchivedEvents·config·values 구조. 창별 자료 집합 대응은 특정되지 않음

```
PS C:\Users\user\Desktop\talk_file> Get-Entropy $UA

File                                                                                Size Entropy
----                                                                                ---- -------
.\01_S22Plus_Before_Migration\jp.co.koeitecmo.zone\files\local\offline_userdata  720  7.7202


PS C:\Users\user\Desktop\talk_file> Get-Entropy $UB

File                                                Size Entropy
----                                                ---- -------
.\02_TabS6Lite_Source\files\local\offline_userdata 2832  7.9164
```

그림 11c. offline_userdata의 크기와 Shannon entropy 출력

3.6 같은 길이 파일의 블록 비교

offline_flag는 01·02 모두 160 B이나 SHA-256이 다름. 같은 길이의 파일이 같은 내용을 가진다는 전제를 이 사례에서 반박함. 16 B로 나눈 10개 동일 위치 블록은 모두 False이며, 10×10 전체 교차 비교에서도 공통 블록이 검출되지 않음. 비정렬 오프셋의 부분 문자열 일치 여부나 다른 블록 크기의 결과는 포함하지 않음.

```
PS C:\Users\user\Desktop\talk_file> $FA = Join-Path $localA "offline_flag"
PS C:\Users\user\Desktop\talk_file> $FB = Join-Path $local  "offline_flag"
PS C:\Users\user\Desktop\talk_file>
PS C:\Users\user\Desktop\talk_file> Get-FileHash $FA -Algorithm SHA256

Algorithm       Hash
---------       ----
SHA256          4BE2F12F5A1B509232A0EDA4947E8C0CB64A3BD6B02ABE36B4C9A3461B9017D8


PS C:\Users\user\Desktop\talk_file> Get-FileHash $FB -Algorithm SHA256

Algorithm       Hash
---------       ----
SHA256          895DFA25AFAECEFD7FD5ACA8499A98F173198A8BAD00917DAC0F96EEA8E762B5
```

그림 2. 160 B offline_flag 두 파일의 SHA-256 불일치와 경로 변수

```
Block  Same
-----  ----
    0 False
    1 False
    2 False
    3 False
    4 False
    5 False
    6 False
    7 False
    8 False
    9 False
```

그림 3. offline_flag의 동일 위치 Block 0~9가 모두 False로 출력됨

```
PS C:\Users\user\Desktop\talk_file>
PS C:\Users\user\Desktop\talk_file> if ($matches) {
>>     $matches | Format-Table -AutoSize
>> } else {
>>     "공통 16바이트 블록 없음"
>> }
공통 16바이트 블록 없음
```

그림 4. 전체 교차 비교에서 공통 16바이트 블록 없음 출력

4 논의

4 1 가설의 판단과 반대 결과

질문	근거	판단
H1 데이터 적용 후 상태 복원	과거 이식 성공 기록 1건. 그림 7~10은 작업·표시 상태의 보조 증적.	사례 내 지지. 통제된 인과 효과나 성공률은 미산출.
같은 파일 길이는 같은 내용을 뜻하는가	offline_flag 160/160 B. 그림 2 해시 불일치, 그림 3 10/10 불일치, 그림 4 공통 블록 0.	이 사례에서 성립하지 않음.
두 자료의 모든 파일이 다르거나 커지는가	일부 동일 경로 해시 일치. flag 크기 동일. login −16 B.	성립하지 않음. 일치·감소 결과도 함께 수록.
16 B 정렬과 높은 엔트로피가 AES를 입증하는가	9쌍 크기 정렬 및 userdata 엔트로피 2개.	알고리즘 판정 불가.

복원과 바이트 동일성은 서로 다른 관찰임. 사용 가능한 상태가 유지되어도 파일의 일부가 다를 수 있으나, 현재 01·02는 통제된 적용 전후 자료가 아니므로 이식이 파일 차이를 발생시켰다고 결론 내리지 않음. 차이의 가능한 설명으로 계정·진행 상태 차이, 앱 실행 중 갱신, 수집 시점·범위 차이, 데이터 변환을 검토함. 이를 구분할 시점별 원본과 실행 기록이 없어 원인 우선순위는 부여하지 않음.

4 2 알려진 사실과 사례의 기여

공식 문서로 알려진 사실은 Android 저장소 접근 제한과 Shizuku의 권한 중개 구조임 [1][2]. 엔트로피 기반 바이너리 분석은 선행 실험 연구가 존재하나 단일 통계량의 해석에는 대상과 조건이 필요함 [3]. OWASP MASVS는 저장·플랫폼 상호작용·분석 및 변조 저항성을 검토하는 보안 기준으로 활용함 [4].

본 사례의 기여는 서비스 종료 게임의 로컬 상태 보존 수행 기록을 권한 전제조건과 구분하여 정리하고, 9개 파일 쌍의 정량 차이와 동일 크기 flag의 블록 불일치를 증적으로 연결한 것임. 새로운 플랫폼 취약점이나 암호 알고리즘 발견을 주장하지 않음.

4 3 보안 영향과 한계

기존 데이터 확보와 ADB 수준 파일 작업이 가능한 환경에서 로컬 상태의 기기 간 재사용 기록을 확인함. 실제 민감정보 노출, 의도적 상태 변조 허용, 무결성 검증 우회 및 서버 계정 소유권 영향은 입증되지 않아 위험도나 CVSS 점수를 부여하지 않음. 보호가 필요한 데이터의 민감도·파일별 역할·키 처리·변조 시 동작을 먼저 조사하고, 기기 바인딩이 필요한 서비스라면 승인된 이전·복구 절차를 함께 설계할 필요가 있음.

주요 한계는 과거 이식 1건, 불완전한 환경·수집 기록, 원본 바이너리 부재, 화면의 기기 대응과 동일 Player ID 전후 대조 부족임. 10개 블록과 100쌍 비교는 같은 파일에 대한 계산으로 독립 반복이 아님. 다른 게임·OS에 대한 일반적인 복원 가능성, 암호화·인증·서버 검증 구현은 이 연구의 결론 범위 밖임.

5 결론

서비스 종료된 Unity 기반 Android 게임에서 리소스 추출과 로컬 데이터의 기기 간 이식 후 기존 상태 복원 기록을 확인함. H1은 해당 사례의 수행 기록에서 지지되지만, 독립 반복과 전후 동일 계정 대조가 없어 인과 효과의 크기나 재현 성공률은 판단할 수 없음. PC 자료 집합의 offline_* 9쌍에서 8쌍의 크기 차이와 동일 크기 offline_flag의 내용 차이를 확인함. 16바이트 정렬, 높은 엔트로피, 블록 불일치는 바이너리 특성에 대한 관찰이며 암호화 알고리즘이나 플랫폼 취약점의 증거로 확대하지 않음. 보존 목적의 이식 가능성과 보안 보호 기능의 유무를 분리하여 평가하는 것이 본 연구의 결론임.

참고문헌

[1] Google. (2026). Storage updates in Android 11. Android Developers. 최종 갱신 2026.10.01.

연결: 일반 파일 접근 조건과 권한 전제조건을 설명함. H1의 상태 복원 자체를 입증하는 문헌은 아님.

[2] Shizuku 프로젝트. (2022). Introduction. 공식 문서. 최종 갱신 2022.08.23.

연결: ADB 수준 권한 중개를 설명하여 데이터 적용 방법의 기술적 배경을 제공함.

[3] Menéndez, H. D., Bhattacharya, S., Clark, D., & Barr, E. T. (2019). The arms race: Adversarial search defeats entropy used to detect malware. Expert Systems with Applications, 118, 246-260.

연결: 바이너리 엔트로피를 이용한 검출·회피 실험이 있는 선행 연구임. H1과 별개로 파일 통계의 해석 및 한계에 연결함.

[4] Holguera, C., & Schleier, S. (2023). MASVS v2.0.0 Release. OWASP Foundation. 2023.04.01.

연결: 저장 보호·플랫폼 상호작용·분석 및 변조 저항성 논의의 기준으로 활용함. 버전별 준수 판정은 실시하지 않음.

문헌 확인일 2026.10.03. 공식 기관·프로젝트 페이지 및 UCL 원문 PDF를 열어 제목·저자·연도와 관련 내용을 확인함. [3]은 초록·서론의 실험 설명을 확인한 후 방법과 해석의 배경으로 채택함. 웹 문서의 갱신 연도, 기준 발표 연도와 연구 수행 시점을 구분함.

재현 방법

제출 ZIP을 빈 폴더에 풀고 Python 3.9 이상에서 scripts/verify_observations.py를 실행함. 외부 패키지와 네트워크 접속이 필요하지 않음. 정상 종료하면 generated/verification.json과 generated/file_size_results.csv가 생성됨. 파일 쌍 9개, 엔트로피 기록 2개, flag 동일 위치 블록 10개, 길이 동일 파일 1개와 차이 파일 8개, 16 B 정렬 및 해시 불일치를 확인함.

실행 명령 python scripts/verify_observations.py

이 절차는 전사 수치의 산술 검산과 패키지 증적의 해시 점검을 재현함. 원본 바이트를 읽어 엔트로피·SHA-256·블록 결과를 다시 계산하는 절차와 구분함. 게임 원본은 패키지에 포함되지 않음.

원본을 별도로 확보한 경우 scripts/analyze_originals.py에 01과 02의 files/local 폴더를 지정함. 파일 크기·해시·엔트로피와 flag 블록 비교를 읽기 전용으로 출력함. 원본별 취득 시점·범위·실행 상태를 먼저 기록하고, 결과가 기존 수치와 다르면 새 결과를 보존함.

명령 예 python scripts/analyze_originals.py --left LEFT_LOCAL --right RIGHT_LOCAL --output NEW_OUTPUT

폴더 또는 파일	내용
paper	최종 논문 DOCX·PDF
data	캡처·비교 문서의 수치 전사 CSV 및 자료 설명
evidence	PowerShell 원본 4개와 표시용 크롭, 폴더·복사·마스킹 게임 상태 화면
scripts	수치 검산, 원본 바이너리 분석 스크립트
source	기존 로컬 데이터 비교 분석 문서
README.md	실행 절차와 재현 범위
criteria_audit.csv	과제 C01~C51 대응 위치와 충족 상태
AI_and_my_judgment.txt	AI 역할과 직접 수행·판단한 내용 3줄

AI와 나의 판단

AI에 맡긴 일: 기존 기록의 논문 구조화, 후보 질문과 사후 가설 정리, 문헌 확인 보조, 수치 검산 및 재현 스크립트 작성.
직접 수행한 일: AssetStudio 리소스 분석, Android 파일 작업과 데이터 이식, 게임 상태 복원 확인 및 비교 자료 확보.
직접 정한 판단 기준: 자료에 없는 사실을 추가하지 않고, 복원 기록과 바이너리 차이를 보존하며, 암호화 방식과 취약점은 근거 없이 확정하지 않음.

부록 연구 질문과 문헌 검토 기록

아래 질문은 과제 문서화 단계에서 AI가 제안하고 현재 근거로 검증 가능한 범위를 평가한 질문임. 과거 수행 전에 이 목록을 작성했다는 의미는 아님. 반대 결과도 기록하며 실제 결과에 맞춰 H1 문장을 수정한 이력은 없음. 초기의 포괄적 관심사인 로컬 데이터 보호를 데이터 적용과 표시 상태 변화로 좁힘.

번호	검토 질문	현재 자료의 판단
1	기존 데이터 적용이 표시 상태에 영향을 주는가	최종 H1. 기록으로 검토 가능.
2	AssetBundle에서 리소스를 추출할 수 있는가	과거 추출 기록 있음. 직접 실행 캡처 부족.
3	기기별 offline_*의 크기는 같은가	9쌍 비교 가능.
4	길이가 같은 flag의 내용도 같은가	해시와 블록 비교 가능.
5	userdata의 바이트 분포는 같은가	엔트로피 기록 2개 비교 가능.
6	파일 크기는 16 B로 정렬되는가	18개 크기 검산 가능.
7	일반 파일 접근과 ADB 권한 작업은 같은가	권한 전제조건 구분 가능. 화면 부족.
8	이식 가능한 파일은 어떤 것인가	파일별 적용 실험 없음.
9	로컬 데이터는 AES로 암호화되었는가	암호 코드·키 자료 없음.
10	변조 데이터는 검출되는가	변조 실험 없음.

AI와 가설을 다듬는 과정에서 측정 가능성, 수집 가능 범위, 반대 결과의 의미를 검토함. “계정 복원이 가능한가”를 데이터 적용이라는 조건과 표시 상태라는 관찰값으로 분리함. 이번 과제에서 추가 데이터를 새로 모은 것으로 쓰지 않고 기존 자료 집합을 재분석하는 범위를 선택함.

문헌 선정과 제외

엔트로피 선행 연구 후보인 Lyda와 Hamrock의 2007년 Using Entropy Analysis to Find Encrypted and Packed Malware는 검색 결과에서 제목·저자를 확인했으나 검토한 원문 링크가 접근 제한 및 브라우저 확인 화면을 반환하여 최종 목록에서 제외함. 원문과 실험 설명을 공개 열람할 수 있는 Menéndez 외 [3]를 채택함. OWASP [4]는 공식 발표 기록에서 저자·연도·제목을 확인함. KISA PRNG·민감정보 노출 및 IBM 조사 사례는 본 게임의 보호 구현을 직접 검증하는 자료가 아니므로 기존 배경 자료에서 본 논문의 논거 목록으로 옮기지 않음.

참고문헌은 학술지·기관·공식 문서명으로 식별하며 제출물에 외부 URL을 사용하지 않음. 원문 열람을 통한 존재와 관련 내용 확인 기록은 위에 정리함.